

SIH26054 DRDO · SOFTWARE · ROBOTICS & DRONES

Engine Twin War Room

Everything I could find on the DRDO aero-piston-engine digital twin problem, and an opinionated plan for building the version that actually beats the other forty teams.

PROBLEM ID	ORGANISATION	PS DEADLINE	GRAND FINALE
SIH26054	DRDO	30 Sep 2026	Dec 2026 · 36 h
TEAM			
6 · 1 F min.			

00 The short version

Almost every team that picks this problem will build the same thing: a random-number generator feeding a CSV, an LSTM trained on it, and a dashboard with red and green lights. It demos fine. It loses, because a DRDO evaluator has seen forty of them by lunch and none of them is a digital twin — they're dashboards with a model bolted on.

A threshold system compares a sensor to a redline. A digital twin compares a sensor to what physics says it *should* read at this exact operating point. The gap between those two numbers is the entire product.

That reframing is free, it's what the problem statement literally asks for in its own background text, and everything worth building follows from it. Three things separate a winning build here:

1. Residuals, not raw signals

Every model downstream eats *residuals* — measured minus twin-predicted — instead of raw sensor values. It makes the anomaly detector immune to throttle changes and altitude, which is the single biggest reason naive builds false-alarm constantly.

2. Faults injected into physics

Faults are parameter changes in the engine model (injector discharge coefficient, ring gap, compressor efficiency), not spikes pasted onto a CSV. The signatures then propagate correctly, and you get exact ground-truth RUL labels for free.

3. A decision, not an alert

"Mission reliability enhancement" is in the title. Everyone shows a fault alert. You show a go / derate / return-to-base call with a probability attached. That's the slide people remember.

THE DEMO MOMENT WORTH PLANNING THE WHOLE BUILD AROUND

Inject a slow **drift on the CHT sensor**, not the engine. Your system reports *"CHT sensor 3 drifting +0.4 °C/min — engine healthy, no corroborating residual on EGT or oil temperature."* A threshold system aborts the mission for a failed forty-dollar sensor. Nobody else in the room will separate sensor faults from engine faults, and the problem statement explicitly lists sensor drift as a fault class. Build for this scene.

01 The problem statement, decoded

Official title: "AI-Enabled Real-Time Digital Twin System for Health Monitoring, Fault Prediction and Mission Reliability Enhancement of Aero Piston Engines used in MALE UAVs." DRDO submitted seven statements this year (SIH26049–26055); this is the only software one in the drones theme.

The background paragraph is the most useful sentence DRDO wrote, and it's the one most teams skip: *"Conventional engine monitoring systems used in UAVs are primarily threshold-based and reactive in nature."* That is DRDO telling you exactly what to beat. Your pitch should open by beating it on

screen, not by describing your stack.

Six required components, and who owns each

The expected solution is lettered A–F. Map your repo to those letters literally — folder names, slide headings, demo order. Evaluators score against a rubric derived from this list, and making the mapping obvious is close to free marks.

§	DRDO ASKS FOR	YOU BUILD	OWNER	CUT IF SHORT?
A	Digital twin core, synchronised with live data, modular	Mean-value engine model of a turbo aero-piston engine running in lockstep with the telemetry stream, engine parameters in a YAML profile so a second engine is a config file, not a rewrite	BE-1	Never
B	Health monitoring: RPM, CHT, EGT, oil, fuel flow, vibration, <i>battery/alternator, injection timing</i>	Per-cylinder CHT/EGT, oil P&T, fuel flow and rail pressure, MAP, turbo speed, vibration RMS + order spectrum, bus voltage and alternator load, injection timing advance	BE-1	Never
C	Fault detection: misfire, injector, lubrication, <i>sensor drift</i> , combustion instability, overheating, vibration	Ten-fault physics-injected library + residual signature matrix for isolation (§05)	BE-1 / BE-2	Ship all 10
D	AI/ML: anomaly detection, RUL, trend analysis, maintenance advice	Physics-guided residual estimator → LSTM autoencoder → fault classifier → two-headed RUL with uncertainty band (§07)	BE-2	Never
E	Simulation & replay: mission replay, environment simulation, high-altitude and endurance scenarios	Timeline scrubber driving the whole UI, ISA atmosphere model, Leh-profile and 18-hour-endurance presets	Dev	Keep replay
F	Dashboard: health, alerts, efficiency trends, advisories, mission reports	React GCS with a live 3D engine that lights the faulted cylinder, plus an auto-generated post-flight PDF	Dev	Never

PS component → what you actually ship. Owner column assumes the 6-person split in §12.

FIVE THINGS IN THE OFFICIAL TEXT THAT MOST BRIEFS MISS

- CAN bus interfaces are named explicitly. Not MOTT — CAN. Real UAV

engine ECUs speak CAN, and showing a CAN frame decoder is a credibility signal MQTT alone doesn't give you.

- **Sensor drift** is listed as a fault class alongside engine faults. This is the sensor-vs-engine discrimination demo. Huge, and almost free once you have residuals.
- **Battery / alternator health and injection timing** are in the monitoring list. Two extra channels, five minutes of work, and they prove you read the actual text.
- **Edge computing and cloud analytics** — a split architecture, not one server. There's a real engineering reason for it (\$06) and stating that reason well is worth more than the code.
- **Deployment roadmap** is a listed deliverable. Most teams ship zero slides on this. It's cheap and it's what a DRDO panel, who are procurement people as much as engineers, actually cares about.

02 Ground truth: which engine you're twinning

Pick a real engine and say its name. "A generic piston engine" reads as a student project; a named engine with published limits reads as engineering. Two good options, and I'd default to the first.

Option A — the VRDE 180 hp aero-diesel (recommended)

DRDO's own Vehicles Research and Development Establishment built this for UAVs. It holds **180 hp constant to 11,000 ft**, runs an indigenously developed **FADEC**, has logged **1,100+ hours** on test profiles, and was trialled at Leh and Changla up to **17,664 ft**. TAPAS BH-201 production spec calls for two VRDE engines at 220 hp each, against 100 hp prototype units, with a demonstrated 28,000 ft ceiling and 18-hour endurance, targeting 30,000 ft and 24+ hours.

Twinning a DRDO engine, in front of DRDO, using DRDO's own published altitude trial numbers as your environment presets, is the cheapest credibility you will ever buy. That 11,000 ft figure is your model's **critical altitude** — the point above which the turbo can no longer hold rated manifold pressure and power starts lapsing. Free physics, free demo scenario, and it's their number.

Option B — Rotax 914-class (the documented fallback)

Option B – Rotax 914 class (the documented factory)

Turbocharged four-cylinder boxer, the workhorse of tactical and MALE UAVs worldwide, and — critically — its operating limits are published in the EASA type certificate, so every number in your model is citable. Use it if you want a spark-ignition profile or if a judge asks for validation data you can point at.

PARAMETER	LIMIT	WHY IT MATTERS TO THE TWIN
Take-off speed (max 5 min)	5 800 rpm	Time-limited rating — your mission model must track cumulative time at rating
Max continuous speed	5 500 rpm	Nominal cruise anchor for the healthy-engine training set
Take-off power (5 min)	84.5 kW	Upper bound on the indicated-power balance
Max continuous power	73.5 kW	Baseline for BSFC and degradation-rate scaling
Cylinder head temperature	135 °C	The classic threshold alarm. Your twin fires ~15 min earlier (§04)
Coolant exit temperature	120 °C	Drives the cooling-degradation fault
Oil temperature (inlet)	130 °C	Second-order confirmation channel for lubrication faults
Oil pressure > 3 500 rpm	2.0 – 5.0 bar	Primary channel for oil leak / pump wear
Oil pressure < 3 500 rpm	0.8 bar min	Speed-dependent limit — a fixed threshold gets this wrong, a twin doesn't
Oil pressure, cold start	7.0 bar max	Startup transient your model should reproduce
Airbox temperature	88 °C max	Intercooler / charge-air health
Manifold pressure, take-off	1 350 hPa	Boost target — turbo degradation shows as inability to hold it
Manifold pressure, max cont.	1 200 hPa	Cruise boost setpoint
Fuel pressure	airbox + 0.25 bar	Referenced to airbox, not ambient — a nice detail to say out loud

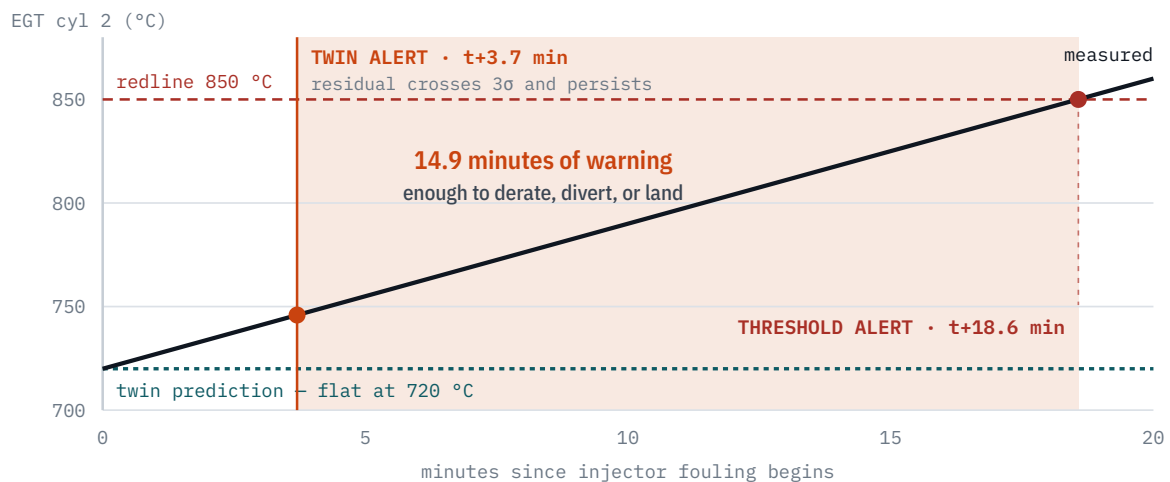
Rotax 914 F/UL certified limits, from EASA TCDS E.122. Use these verbatim as your model's redlines and as the "threshold system" your twin beats.

Notice the TCDS publishes no EGT limit. That's real and worth mentioning:
EGT on these engines is a *trend* parameter. not a redline parameter — which

is precisely the argument for a twin that models expected EGT rather than comparing it to a number that doesn't exist.

03 The one chart that wins the room

Show this before you show your architecture. It is the whole argument in one picture: a slowly clogging injector on cylinder 2, exhaust gas temperature creeping up, and two systems watching it. The redline system is silent for fifteen minutes. The twin is not, because the twin knows EGT on cylinder 2 has no business being 12 °C above its three siblings at this fuel flow and this altitude.



Simulator output, not a claim about a physical engine — and say exactly that when you present it. Baseline 720 °C, fouling ramp 7 °C/min, residual noise $\sigma = 4$ °C, alert requires 3σ sustained over a 2-minute persistence window.

The honesty note in that caption matters more than you'd think. Volunteering the limits of your own data is how you stop a judge from "catching" you with it, and it costs you nothing because the mechanism is real even if the numbers are synthetic.

04 Architecture

Six layers, each independently demoable. That last property matters on day five when something breaks: you want to be able to drop a layer and still have a story.

LAYER 0 · TRUTH

Mean-value engine model + fault injector

manifold · crank · turbo · thermal states
10 faults as parameter perturbations

LAYER 1 · TRANSPORT

CAN bus → MQTT → WebSocket

SocketCAN vcan0, CANaerospace-style frames
fast lane 20 kHz on-board · slow lane 10 Hz down

LAYER 2 · EDGE (ON THE UAV)

Residual estimator + anomaly detector

M1 physics-guided predictor · M2 LSTM autoencoder
vibration order features · runs at 10 Hz

LAYER 3 · GROUND ANALYTICS

Isolation, classification, remaining useful life

M3 classifier + signature matrix · RUL p10/p50/p90
damage integrator · fleet history in Parquet

LAYER 4 · DECISION

Mission reliability engine

forward-propagate twin over remaining route
P(complete) · derate advisory · RTB call

LAYER 5 · GROUND CONTROL STATION

React + three.js dashboard, replay scrubber

3D engine lights the faulted cylinder
post-flight PDF · mission risk map

layers 0-1 replaced by a real ECU and a real datalink – nothing above them changes

The bottom two layers are the only ones that know the data is simulated. That boundary is your deployment story, and it is worth a slide of its own.

05 Layer 0 – the physics, and why it's the moat

A mean-value engine model is the right fidelity here. It ignores what happens inside a single crank rotation and tracks cycle-averaged states, which is exactly the level a health monitor needs and cheap enough to run faster than real time. Five state variables carry the whole engine.

The five states

1. Intake manifold pressure

$$dp_{tk}/dt = (R \cdot T_{tk} / V_{tk}) \cdot (\dot{m}_{3\ o\ mnk} - \dot{m}_{9\ n-})$$

Filling and emptying. Compressor pushes air in, the engine pulls it out. Turbo degradation shows up here first.

2. Air ingested (speed-density, 4-stroke)

$$\dot{m}_{g_{n-}} = \eta_k \cdot (p_{tk} / R \cdot T_{tk}) \cdot V_a \cdot N / 120$$

N in rpm, V_a swept volume. The 120 is 2 revolutions per cycle $\times$ 60 s. Volumetric efficiency η_k is where ring wear and valve recession enter the model.

3. Crankshaft dynamics

$$J \cdot d\omega/dt = T_{x_{na}} - T_{=h_{x-}} - T_{lmhk} - T_{x_{ha}} =$$

$$T_{x_{na}} = \eta_x \cdot \dot{m}_= \cdot Q_{kht} / \omega$$

Per-cylinder torque contributions summed. Kill one cylinder's contribution and you get a real misfire signature, not a pasted-on spike.

4. Cylinder head temperature (lumped, per cylinder)

$$m \cdot c_h \cdot dT_{+ht}/dt = \dot{Q}_{g_{askk}} - h_{a \times m} \cdot A_{=x_{\vartheta}} \cdot (T_{+ht} - T_{as-})$$

Four independent thermal states. This is what makes per-cylinder fault localisation physically meaningful instead of a colour you assigned arbitrarily.

5. Turbocharger shaft

$$J_{t+} \cdot \omega_{t+} \cdot d\omega_{t+}/dt = \eta_s \cdot P_{t_{\square k-}} - P_{+hsk}$$

$$P_{+hsk} = \dot{m}_{g_{n-}} \cdot c_h \cdot T_1 / \eta_+ \cdot [(p_2/p_1)^{(\gamma-1)/\gamma} - 1]$$

Scale η_+ down over time and you get turbo degradation that only reveals itself above critical altitude – a beautiful demo, because it is invisible at sea level.

Atmosphere and altitude

Standard ISA troposphere, three lines of code, and it unlocks every "environmental condition simulation" the PS asks for:

$$T = 288.15 - 0.0065 \cdot h \quad [K]$$

$$p = 101325 \cdot (1 - 2.25577 \times 10^{-5} \cdot h)^{5.2559} \quad [Pa]$$

Then add a temperature offset for ISA+20 (hot-and-high, the Leh case) and a humidity term if you want to show off. Above critical altitude the turbo can't hold boost, power lapses, and everything downstream reacts on its own.

Degradation with real ground truth

Degradation with real ground truth

This is the part that turns a data generator into a dataset with labels. Wear accumulates by physically motivated laws, and the model carries an explicit damage state D from 0 to 1:

Mechanical wear (Archard-type): $dD/dt \propto k \cdot F_n \cdot v$
Thermal ageing (Arrhenius): $dD/dt \propto \exp(-E_a / R \cdot T_{ht})$
Cumulative damage (Miner): $D = \sum n_x / N_x$, failure at $D = 1$
Ground-truth RUL = $(1 - D) / (dD/dt)$

Because you own D , every training sample has an exact RUL label — no censoring, no guessing. This is how NASA generated C-MAPSS, and saying so out loud is a strong move.

SAY THIS WHEN A JUDGE SAYS "BUT YOUR DATA IS FAKE"

"It's synthetic, and so is C-MAPSS — the benchmark this entire field has been ranked on for eighteen years. Our fault signatures come from published FMEA literature, our limits from the engine's type certificate, and the pipeline is dataset-agnostic. Here it is running unmodified on NASA's N-CMAPSS flight-condition data." Then show it. That last sentence is worth building for.

06 The fault library and the signature matrix

Ten faults, each a parameter perturbation with its own time evolution. Because they act through the physics, the signatures come out consistent for free — you never have to hand-author what a fault "looks like."

FAULT	INJECTED AS	RPM	EGT _I	CHT _I	FUEL	MAP	OIL P	OIL T	VIB	RIPPLE
Injector fouling, cyl <i>i</i>	C _o ↓ 0.4 %/min	.	▲▲	▲	▼	.	.	.	▲	▲▲
Ignition / misfire, cyl <i>i</i>	skip combustion event	▼	▼▼	▼	.	▲	.	.	▲▲	▲▲
Piston ring wear / blow-by	η _k ↓, blow-by ↑	▼	▲	▼	▲	▲	▼	▲	▲	.
Oil leak / pump wear	pump gain ↓	.	.	▲	.	.	▼▼	▲▲	▲	.

Turbo degradation	$\eta_+ \downarrow 8\%$	▼	▲	▲	•	▼▼	•	•	▲	•
Cooling / intercooler fouling	$h \cdot A \downarrow 15\%$	•	•	▲▲	•	•	•	▲	•	•
Detonation / pre- ignition	knock model armed	•	▼	▲▲	•	•	•	▲	▲▲	•
Fuel filter clog / vapour lock	rail p ↓ with altitude	▼	▲	▲	▼▼	•	•	•	•	▲
Bearing wear	friction torque ↑	▼	•	•	▲	•	▼	▲▲	▲▲	•
Sensor drift, channel x	bias ramp on x only	•	▲▲	•	•	•	•	•	•	•

Signature matrix. ▲▲ strong rise, ▲ mild rise, ▼ fall, • unchanged. Ripple = 0.5-order component of crank angular velocity. This table is simultaneously your fault library spec, your isolation algorithm, and one of your best slides.

Read the last row against the first. An injector clog moves *five* residuals in a physically coupled pattern. A drifting EGT sensor moves *one*, with nothing corroborating it — no ripple, no CHT response, no fuel-flow change. That's the entire sensor-versus-engine discriminator, and it falls out of the matrix without any extra model. Cosine similarity between the live residual vector and each row gives you an explainable diagnosis with a confidence number attached.

WHY THE RIPPLE COLUMN EARNS ITS PLACE

In a four-stroke, any one cylinder fires once per 720° of crank rotation — so a single-cylinder problem shows up at **0.5 engine order** in the crank angular velocity spectrum. That's the standard automotive misfire-detection trick, it's in patents from the early nineties, and it localises the fault to a cylinder from a single existing sensor. Cheap to simulate, and it sounds like you know engines.

07 Layer 1 — transport, and the argument for edge

The PS names CAN bus, edge computing and cloud analytics. Those three are one story, and it's a story about bandwidth.

Knock and bearing signatures live at 5–20 kHz. A MALE UAV's downlink to the ground station is a few tens of kilobits per second over a satcom or line-of-sight link shared with video and command traffic. You cannot send raw vibration down that pipe, ever. So the fast signal processing has to happen on the engine, and only the extracted features and health state go down. That is not an architecture preference — it's arithmetic, and stating it that way makes your edge/cloud split sound inevitable instead of trendy.

LANE	RATE	CHANNELS	LIVES WHERE	DOWN THE LINK?
Fast	5–20 kHz	Tri-axial vibration, crank angular position, knock sensor	On-board only	No — features only
Slow	10–50 Hz	RPM, MAP, 4×CHT, 4×EGT, oil P/T, fuel flow, rail pressure, injection timing, bus voltage, alternator current, TCU state	Bus → ground	Yes
Health	1 Hz	Health index, residual vector, fault probabilities, RUL band, mission risk	Computed at edge	Yes — a few hundred bytes/s

Two-lane telemetry. The split is the design.

Make the CAN layer real — it's a 30-minute job

Linux ships a virtual CAN interface. Bring one up, publish your telemetry as actual CAN frames, and decode them on the other side. Having `candump vcan0` scrolling in a corner of your demo screen does more for credibility than any slide, because it says "this speaks the bus the ECU speaks."

```
sudo modprobe vcan
sudo ip link add dev vcan0 type vcan
sudo ip link set up vcan0
pip install python-can paho-mqtt

# CANaerospace-style: 4-byte header (node, type, service, msg code)
# + 4-byte payload, identifiers in the 300–1799 range
frame = can.Message(arbitration_id=0x12C,          # 300 = EGT cyl 1
                    data=struct.pack('>BBBBf', node, DT_FLOAT, 0, seq, egt1
```

Then bridge CAN → MQTT for the ground side, and MQTT → FastAPI WebSocket for the browser. Three hops, each of which is what the real system would use. Name-drop CANaerospace (Stock Flight Systems, 1998; adopted by NASA as the AGATE databus standard in 2001; identifiers 300–1799 reserved)

and **DroneCAN / Cyphal** (what ArduPilot and PX4 actually run, 1 Mbit/s typical) — two sentences that tell an evaluator you looked at the real ecosystem.

Storage: use Parquet files plus DuckDB for history. It's ten minutes of setup, queries fly, and mission replay becomes a range scan. Put TimescaleDB in the deployment roadmap slide, not in the hackathon repo — you get the credit without the setup cost.

08 Layers 2–3 — three models, no more

The temptation is to build five model families because the PS mentions several. Resist it. Three models, each with one job, each of which you can explain in a sentence.

M1 · Physics-guided residual estimator

Job: predict what every sensor should read at this operating point, plus the ones you can't measure at all.

Shape: $\hat{y} = f_{\text{physics}}(u) + g_{\text{NN}}(u)$.

The MVEM does the heavy lifting; a small MLP learns the correction. Because the network only learns a delta, it trains on very little data and never goes insane at an altitude it hasn't seen.

Virtual sensors it gives you: peak cylinder pressure, knock margin, per-cylinder combustion efficiency, turbo shaft speed, oil film thickness. DRDO cares about virtual sensing specifically — these are the payoff.

M2 · LSTM autoencoder on residuals

Job: flag "this doesn't look like a healthy engine" without being told what the fault is.

The critical detail: it eats the *residual* window, not raw sensors. An autoencoder on raw signals just learns the flight profile and screams every time the throttle moves. On residuals it learns "healthy means near-zero and white," which is operating-point invariant.

Threshold: 99.5th percentile of reconstruction error on healthy validation data, plus a persistence rule (N of M windows). Never a hand-picked constant.

M3 · Isolation + remaining useful life

Classifier: 1D-CNN or GRU over the

Metrics to put on the slide

RMSE alone looks like a course

residual window → softmax across the ten fault classes. Labels are free because you generated the faults.

Paired with the signature matrix from §06 for the explanation. Show both agreeing: the network gives accuracy, the matrix gives the *why*. Judges remember the why.

RUL, two heads: a quantile-regression network reporting p10/p50/p90, and a physics head that integrates the damage rate forward. Report both. "Physics says 41 h, the network says 37, we advise on 37."

assignment. Add the field's own measures: the NASA **asymmetric scoring function** (late predictions punished harder than early ones), **prognostic horizon**, and α - λ accuracy. For detection, report **lead time** and **false alarms per flight hour** — that second one is what a maintenance officer actually asks about, and quoting it unprompted is a strong signal.

Straight talk on PINNs and GNNs

You mentioned both. Here's where I'd actually spend the time.

PINN — do the soft version, not the hard one. A true PINN with PDE residuals in the loss is a research project that may simply not converge in your window, and if it doesn't you have nothing. The practical version delivers 90% of the credit at 10% of the risk: keep the physics baseline as the model's backbone (that's already "physics-informed" in the way that matters) and add soft constraint terms to the loss:

$$L = L_{\text{data}} + \lambda_1 \cdot L_{\text{fuel}} + \lambda_2 \cdot L_{\text{sa}} + \lambda_3 \cdot L_{\text{shnhthn}}$$

L_{fuel} : fuel power in = brake power + exhaust enthalpy + heat rejected + friction

$$L_{\text{sa}} : \dot{m}_{\text{ax}} + \dot{m}_{\text{ex}} = \dot{m}_{\text{in}} - \dot{m}_{\text{ex}} = \dot{m}_{\text{ex}} - \dot{m}_{\text{ex}}$$

L_{shnhthn} : $\partial \text{CHT} / \partial (\text{fuel flow}) \geq 0$ at fixed airflow

That third term costs one gradient penalty and buys you sane behaviour outside the training envelope — which is exactly what you need at 25,000 ft on data you mostly generated at 15,000.

GNN — genuinely the right tool, genuinely a stretch goal. The engine *is* a graph: cylinders hang off a shared manifold, share an oil circuit, share a crank. Nodes as components, edges as physical couplings, node features as local residuals, and message passing localises a root cause through the coupling structure. It is a better fault-propagation model than a flat classifier.

It is also the fourth thing you'd build, and the fourth model family you build. On a six-day plan it lands on day 5 and only if day 4 closed clean (\$12); and if a judge asks why it isn't in the prototype, the answer "we scoped it as phase two because the signature matrix already gives us explainable isolation" is a better answer than a broken GNN.

THE HIGHEST-VALUE 90 MINUTES IN THE WHOLE BUILD

Run your residual → autoencoder → RUL stack, unmodified, on NASA's C-MAPSS or N-CMAPSS data and put the score on a slide. It is the only thing that proves your ML isn't just memorising your own simulator. Both live in the NASA Prognostics Center of Excellence repository — "Turbofan Engine Degradation Simulation" is C-MAPSS, "...Simulation-2" is N-CMAPSS, which is the better one because it carries real flight conditions. Have a teammate do this in parallel from hour one.

09 Layer 4 — the part nobody else will build

"Mission Reliability Enhancement" is in the title of the problem statement and almost every team will read straight past it. Detection is table stakes. A *decision* is the differentiator.

The mechanism is simple once you have a twin: you already have a model of the engine, and you already have a mission profile ahead of you. So run the twin forward, with the degraded parameters, over the remaining route, Monte Carlo over the RUL uncertainty, and count how many runs finish inside limits.

$$P(\text{complete}) = (1/M) \cdot \sum_u c_1^n \cdot \mathbb{1}[\text{twin}(\theta_u, \text{route}_{h-saxm \times m^9}) \text{ stays within limits}]$$

θ_u sampled from the posterior over degradation state. $M = 200$ is plenty and runs in under a second because the MVEM is cheap. This is the payoff for building a fast physics model instead of only a neural net.

Then turn that number into three outputs a mission commander can act on:

- **The call.** *"Continue: 62% · Derate to 78% power: 94% · RTB now: 99%."* Three options, three probabilities, not a red light.

The cost of the safe option. *"Derating costs 40 minutes of loiter on station."*

- **The cost of the safe option.** *Detaching costs 70 minutes of tower on station.* Reliability advice that ignores the mission is useless advice, and showing the trade-off is what makes it read as a defence product rather than a maintenance tool.
- **Point of no return.** Computed from remaining fuel and the *degraded* BSFC, not the book figure. Put it on the map as a moving marker. It is a genuinely striking visual and it falls straight out of the physics you already have.

Render the route with segments coloured by predicted risk, and let the operator drag the planned altitude down to watch the probability move. That interaction — a judge grabbing the mouse and changing the mission themselves — is worth more than any amount of model architecture.

10 Layer 5 — the dashboard, and one contrarian call

React + Vite + TypeScript, Tailwind, `@react-three/fiber` and `drei` for the 3D. That part is uncontroversial. Here's the part that isn't.

Build the 3D engine from primitives. Do not download a model.

Every instinct says find a nice engine GLB on Sketchfab. Don't. A downloaded model arrives with a few hundred unnamed meshes, and you will spend three hours in the console trying to work out which one is cylinder two — and you do not have three hours to spare on that. Meanwhile it's 40 MB and it drags your frame rate.

Instead compose it yourself out of primitives: four cylinders as `cylinderGeometry` with stacked torus fins, a crankcase as a rounded box, the turbo as a torus plus a cone, exhaust as tube geometry along a curve. It takes about ninety minutes, every mesh is a variable you named, and binding health state to appearance becomes trivial:

```
// cylinder colour is a thermal ramp, emissive intensity is the anomaly score
<meshStandardMaterial
  color={thermalRamp(cht[i], 60, 135)} // steel → amber → red
  emissive={faultColour(faultProb[i])}
  emissiveIntensity={0.15 + 0.85 * anomalyScore[i]} />
```

It will also look better, because it will look deliberate rather than like a stock asset dropped into a student project. Animate the crank and pistons at scaled RPM — that single touch is what makes people say "twin" instead of "diagram."

The rest of the screen

- **Strip charts: use uPlot, not Recharts.** Recharts re-renders the React tree per frame and will visibly drop frames at 50 Hz with eight series. uPlot handles tens of thousands of points on canvas without complaint. This is the single most common performance mistake in hackathon dashboards.
- **A residual heatmap** — sensors down, time across, colour as deviation. One glance shows both *that* something is wrong and *which channels* are wrong. It is the most information-dense square on the screen and it costs almost nothing to draw.
- **RUL with its confidence band always visible.** A bare number reads as overconfident to anyone who works with real hardware. A p10/p90 band reads as someone who has thought about it.
- **A timeline scrubber that drives everything, 3D model included.** The PS asks for mission replay explicitly; making the scrubber global rather than a chart control is what turns a feature into a capability.
- **An explain drawer.** Click any alert, see the residual contributions and the matched signature row. This is your interpretability story and it's a UI panel, not a research problem.

Aesthetically: dark ground, tabular numerals everywhere, units labelled on every axis, and no rounded pastel cards. You want it to look like something in a ground control station, not like a SaaS analytics product. Look at QGroundControl and Mission Planner for vocabulary, then make it about ten years newer.

11 The four-minute demo, scripted

Write this before you write the code, and rehearse it until it's boring. The build serves the script, not the other way round.

Twin overlay sits exactly on the measurement. Residuals flat. Say: "every parameter is green, and a threshold system sees exactly what you see. Watch."

0:40**Inject injector fouling on cylinder 2**

Slow ramp. Every limit still green. Residual on EGT₂ lifts within seconds; the anomaly score crosses at t+22 s of simulated time. Cylinder 2 goes amber in the 3D view.

1:20**Isolation and explanation**

"Injector fouling, cylinder 2, 91%." Open the explain drawer: EGT₂ up, fuel flow trending down, 0.5-order ripple present, oil untouched — matched against the signature row. The network and the physics agree.

2:00**Remaining useful life**

3.2 h, p₁₀–p₉₀ band 2.4–4.1 h. Show both heads. Volunteer the uncertainty before anyone asks for it.

2:25**The decision**

Mission panel recomputes live: continue 58%, derate 94%, RTB 99%. Route recolours, point of no return slides. Hand the mouse to a judge and let them drag the cruise altitude.

3:00**The twist — sensor drift on CHT₃**

System reports a *sensor* fault, not an engine fault, and shows why: no corroborating residual anywhere else. "A threshold system just aborted a mission for a forty-dollar thermocouple." This is the line they'll repeat to the next team.

3:30**Replay and report**

Scrub the timeline back through the whole event. Generate the post-flight PDF on screen. Ten seconds, and it closes out PS components E and F.

3:50**Close on deployment, not on tech**

One architecture slide: what runs on a Jetson-class board on the airframe, what runs on the ground, what changes when a real ECU replaces layer 0. Which is nothing above layer 1.

Pre-generate every dataset and ship pre-trained weights in the repo. Never train during a demo, never generate data during a demo. Have the whole

scenario reproducible from a single `make demo`, and record a full video backup the night before. Assume the venue wifi will fail, because at some point it will.

12 Six days, six people

Six days is a completely different game from a twelve-hour sprint, and the danger inverts. In twelve hours you lose by over-scoping. In six days you lose by *spreading* — ten things at 60% on day six, nothing that survives a full demo run. The fix is one rule, and it's the most important line on this page:

Build the twelve-hour version first, finish it by end of day 2, tag it, and record a demo video of it. Everything after that is additive on a base that already works.

That single decision means you can never be caught with nothing. Every day from three onward, you are choosing to improve a working system rather than praying an unfinished one converges. It also means the video you record on day 2 becomes your worst-case submission, and every later recording replaces it.

What six days buys you that twelve hours doesn't

Not more features. **Measurement.** Almost every team at this level ships a prototype with zero numbers attached to it, because they ran out of time before they could measure anything. Six days means you show up with a false-alarm rate, a detection lead time, an ablation, and a benchmark score on somebody else's data. That is what separates you from the other six-day teams, and it's worth more than a fourth model.

01

The transfer result

Your stack running unmodified on NASA N-CMAPSS, with a score. Moves from "nice if we get to it" to **mandatory**. It is the only hard proof your ML isn't

02

The false alarm rate

Run 100 healthy simulated flights and count the spurious alerts. Report per flight hour. Nobody asks a question you've already answered on a slide.

memorising your own simulator.

03

The ablation

Same autoencoder on raw signals vs. on residuals. One bar chart. It proves your central design claim instead of asserting it, and it takes an afternoon.

04

Edge latency, measured

The residual model plus autoencoder on a Jetson, a Pi, or failing that a CPU-only thread with the clock speed noted. A real millisecond figure kills the "can this fly?" question dead.

05

All ten faults

The six-fault cut was a twelve-hour compromise. With six days you ship the full library, including detonation and bearing wear, and a confusion matrix across all of them.

06

The GNN, honestly

Now reachable — but only on day 5, only if day 4 closed clean, and only as a second isolation head next to the signature matrix. Never as a replacement for it.

DAY	YOU (FRONTEND/3D)	BE-1 (PHYSICS)	BE-2 (ML)	R1 / R2 / R3
Day 0 half day	All six, no code. Freeze the JSON telemetry schema, the engine profile YAML, and the demo script from §11 into the repo README. Pick the engine profile (§02). Agree who owns which PS letter A–F. This half day pays for itself three times over.			
Day 1	Vite + Tailwind scaffold, one live chart off a mocked socket	MVEM states 1–3 + ISA atmosphere, publishing over MQTT	Pull N-CMAPSS, get a baseline RUL number on it before touching your own data	R1: PS decode + references. R2: deck skeleton. R3: repo, CI, make targets
Day 2	uPlot strip charts, 3D engine from primitives, health binding	Thermal + turbo states, six faults, healthy dataset generated	M1 residual estimator + M2 autoencoder, thresholds calibrated	R3: first end-to-end dry run. R2: record v1 video
Day 2 pm	Tag v1 and record it. A working vertical slice: simulate → stream → residuals → anomaly → dashboard. It is not impressive yet. It is <i>finished</i> , and from here you cannot lose everything.			
Day 3	Explain drawer, residual heatmap, alert ribbon, replay scrubber	Remaining four faults, damage integrator with ground-truth RUL	M3 classifier + signature matrix + two-headed RUL with p10/p90	R1: judge Q&A sheet from §13. R2: slides 1–5
Day 4	Mission risk map, route colouring, point-of-no-return	Mission forward-propagation, Monte Carlo over	Measurement day: false alarm rate, lead times, ablation, N-	R3: second dry run + bug triage. R1: deployment roadmap

	marker	degradation state	CMAPSS score	slide
Day 5	Visual polish, post-flight PDF, dark-mode pass, empty states	CAN on <code>vcan0</code> , CANaerospace-style frames, <code>candump</code> in the demo	Edge latency benchmark; GNN <i>only</i> if day 4 closed clean	R2: deck done. R3: third dry run, stopwatch it
Day 5 pm	Hard feature freeze. No new code after this point, no exceptions, including yours. Anything unfinished gets reverted to the last working commit, not "quickly fixed."			
Day 6	Rehearsal only. Run the full demo ten times, different person driving each time. Q&A drill from §13 until answers are reflexive. Final video recorded and verified playable offline. Laptops charged, everything mirrored locally, one teammate carrying a duplicate.			

Six days. Everything before the day-2 freeze is about getting a working spine; everything after is about making it undeniable.

THE FAILURE MODE TO WATCH FOR ON DAY 4

Around day four somebody will want to add a transformer, or swap the autoencoder for something newer, or rebuild the frontend "properly." That urge is what six days does to people. The answer is no. Day 4 is measurement day precisely because it is the day the team most wants to start something new, and giving them numbers to produce is the cheapest way to absorb that energy into something a judge will actually reward.

ON THE THREE LESS-TECHNICAL MEMBERS

Over six days they can carry real weight rather than filling time. R1 owns domain research, the PS-to-repo mapping, and the judge Q&A sheet — by day 3 they should be able to explain critical altitude and 0.5 engine order without notes. R2 owns the deck, the script, and a fresh backup video at the end of every single day. R3 owns QA and is the only person with authority to say "that's not in the demo script, don't build it." Over six days that third role is worth more than another developer.

The grand finale in December is 36 hours at a nodal centre. You will arrive with all of this already built, so those 36 hours go on hardware-in-the-loop if you can borrow anything with a CAN interface, a second engine profile to prove portability live, and whatever the finale's own twist turns out to be.

13 The questions that decide it

A DRDO panel will spend more time interrogating you than watching you. Rehearse these until the answers are reflexes. Whoever on the team is weakest technically should be able to give the first three.

? **Your data is synthetic. Why should we believe any of it?**

"Because the mechanism is real even where the numbers are generated. The redlines come from the engine's type certificate, the fault signatures from published failure-mode literature, and the degradation from Archard wear and Miner's rule — not from a random number generator. C-MAPSS, which this whole field has benchmarked on for eighteen years, is synthetic too. And the pipeline doesn't care: here it is running unmodified on NASA's N-CMAPSS flight data, with the score."

? **What does this give us that a threshold alarm doesn't?**

? **How is this a digital twin and not a dashboard with a model behind it?**

? **Which engine is this, and does it transfer?**

? **How does this get onto an actual aircraft?**

? **What's your false alarm rate?**

? **Why should we trust an RUL number from a neural network?**

? **What did you not build, and why?**

14 What to cut, and how teams lose this

Cut without hesitation

- The graph neural network
- Real PDE-residual PINN training
- Any database that isn't Parquet or SQLite
- Auth, user accounts, multi-tenancy

Never cut

- The residual concept — it's the whole thesis
- The sensor-drift discrimination demo
- Per-cylinder resolution in the 3D view

- Docker Compose with six services
- A trained model for anything

- The RUL uncertainty band
- The mission decision panel
- The replay scrubber